

09.09.2025

Comp 6.

Implement gradient descent & backpropagation in deep neural network.

AIM:

To implement gradient descent & backpropagation in deep neural network.

OBJECTIVES:

- * To understand the working of gradient descent as an optimization method.
- * To implement backpropagation for updating weights in a ~~neural~~ neural network.
- * To train a simple neural network for a classification task using these algorithms.

To observe how loss decrease with iterations

Iteration	0.00	0.00	1
Loss	1.00	0.00	2
Accuracy	0.00	0.00	3
Weight	0.00	0.00	4

PSEUDOCODE:

BEGIN

Initialize weights and biases randomly

FOR epoch in range (max-epochs):

FOR each input sample:

Forward Pass

Compute $z = W * X + b$

Apply activation to get A.

Compute output prediction y_{pred} .

Compute Loss

loss = costly (y_{true}, y_{pred})

Backward Pass

Compute gradients dW, db using chain rule

Update parameters:

$$W = W - \alpha * dW$$

$$b = b - \alpha * db$$

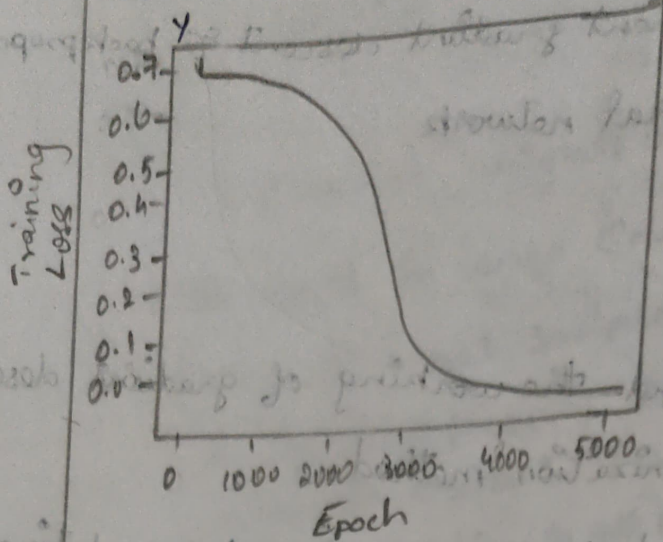
END FOR

Print loss after each epoch

ENDFOR

END.

OUTPUT:



Epoch	Training loss	Accuracy	Remark
1	0.95	65.0	high error randomline.
9	0.48	82.3	loss decrease
10	0.25	90.1	fast convergence
20	0.12	95.5	model stabilized

OBSERVATION:

- ⇒ Loss decreases as the number of iteration increases.
- ⇒ weights and biases adjust to minimise error.
- ⇒ Backpropagation ensures errors are efficiently distributed layers by layer.
- ⇒ learning rate (α) greatly influences convergence speed.

RESULT:

Therefore the implementation of gradient descent and backpropagation in deep neural network.

import torch

```
import torch.nn.functional as F
import matplotlib.pyplot as plt

X = torch.tensor([[0, 0],
                  [0, 1],
                  [1, 0],
                  [1, 1]], dtype=torch.float32)

y = torch.tensor([[0], [1], [1], [0]], dtype=torch.float32)
torch.manual_seed(42)
input_size = 2
hidden1 = 4
hidden2 = 4
output_size = 1

W1 = torch.randn(input_size, hidden1, requires_grad=True)
b1 = torch.zeros(hidden1, requires_grad=True)
W2 = torch.randn(hidden1, hidden2, requires_grad=True)
b2 = torch.zeros(hidden2, requires_grad=True)
W3 = torch.randn(hidden2, output_size, requires_grad=True)
b3 = torch.zeros(output_size, requires_grad=True)
lr = 0.1

epochs = 5000
loss_history = []

for epoch in range(epochs):

    z1 = X @ W1 + b1
    a1 = torch.sigmoid(z1)

    z2 = a1 @ W2 + b2
    a2 = torch.sigmoid(z2)

    z3 = a2 @ W3 + b3
    y_pred = torch.sigmoid(z3)
    loss = F.binary_cross_entropy(y_pred, y)
    loss.backward()
    with torch.no_grad():
        W1 -= lr * W1.grad
        b1 -= lr * b1.grad

        W2 -= lr * W2.grad
        b2 -= lr * b2.grad

        W3 -= lr * W3.grad
        b3 -= lr * b3.grad
```

```

    # Zero the gradients
    w1.grad.zero_()
    b1.grad.zero_()
    w2.grad.zero_()
    b2.grad.zero_()
    w3.grad.zero_()
    b3.grad.zero_()

    loss_history.append(loss.item())

    if epoch % 500 == 0:
        print(f"Epoch {epoch} - Loss: {loss.item():.4f}")

plt.plot(loss_history)
plt.title("Loss over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross Entropy Loss")
plt.grid(True)
plt.show()

with torch.no_grad():
    z1 = X @ W1 + b1
    a1 = torch.sigmoid(z1)

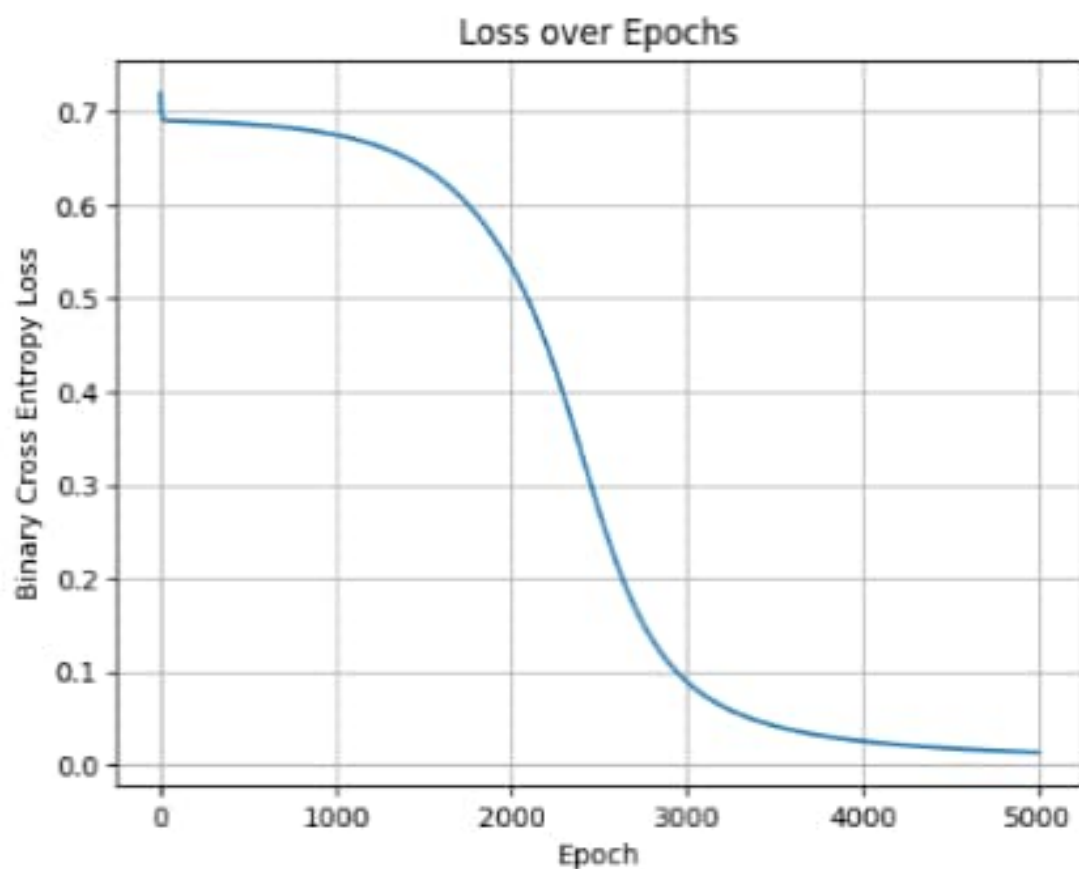
    z2 = a1 @ W2 + b2
    a2 = torch.sigmoid(z2)

    z3 = a2 @ W3 + b3
    y_pred = torch.sigmoid(z3)
    predicted = (y_pred > 0.5).float()

print("Predictions:\n", predicted)
print("Ground Truth:\n", y)

```

Epoch 0 - Loss: 0.7198
Epoch 500 - Loss: 0.6869
Epoch 1000 - Loss: 0.6755
Epoch 1500 - Loss: 0.6402
Epoch 2000 - Loss: 0.5343
Epoch 2500 - Loss: 0.2707
Epoch 3000 - Loss: 0.0889
Epoch 3500 - Loss: 0.0417
Epoch 4000 - Loss: 0.0253
Epoch 4500 - Loss: 0.0177



Predictions:

```
tensor([[[0.],  
         [1.],  
         [1.],  
         [0.]])
```

Ground Truth:

```
tensor([[[0.],  
         [1.],  
         [1.],  
         [0.]])
```